

```
<!--Universidad Veracruzana-->
```

Patrones Builder y Abstract Factory {

```
<Por="Equipo 8"/>
```

```
<Luz Fernanda Herrera Juárez  
Victor Alejandro Gómez Carballo/>
```

```
}
```



Contenidos por cada patrón

01

Problema

02

Patrón

03

Estructura

04

Consecuencias

05

Implementación

06

Demostración

Primer patrón {

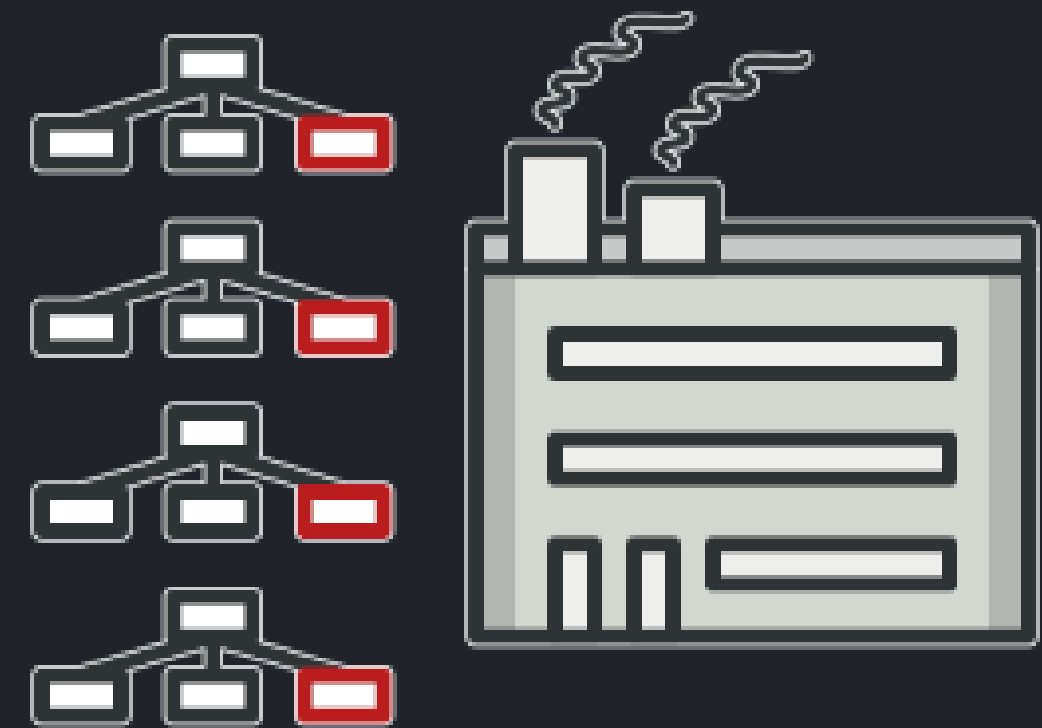
Abstract Factory

}

Problema

El problema que resuelve el patrón Abstract Factory radica en la implementación de clases creadas para objetos específicos.

Cuando surge la necesidad de integrar nuevas formas de construir objetos, la solución inicial comienza a romperse debido a la dependencia de clases concretas.



Recurso gráfico utilizado como apoyo visual.



Solución {

Para resolver esto, el patrón Abstract Factory divide y abstrae la creación de objetos, permitiendo trabajar con diferentes familias de productos sin modificar la lógica principal del sistema.

}

Patrón {

Nombre Oficial

Abstract Factory o Fábrica Abstracta

Clasificación

Creacional

Alias

Kit

Alcance

Objeto

Intención del patrón

" Proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin especificar sus clases concretas ".

Gamma et al., 2003, p. 79.

}

Estructura {

Responsabilidades de cada parte {

- **Fabrica Abstracta** : Interfaz que declara la firma de las operaciones de los productos abstractos que se van a crear
- **Fabrica Concreta** : Implementa las operaciones determinadas por la interfaz para poder crear un producto concreto.
- **Producto Abstracto** : Interfaz define las características para cada tipo de objeto que se va a crear.
- **Producto Concreto** : Define un objeto que va a ser creado por la fábrica correspondiente.
- **Cliente**: Utiliza las interfaces abstractas.

}

Estructura {

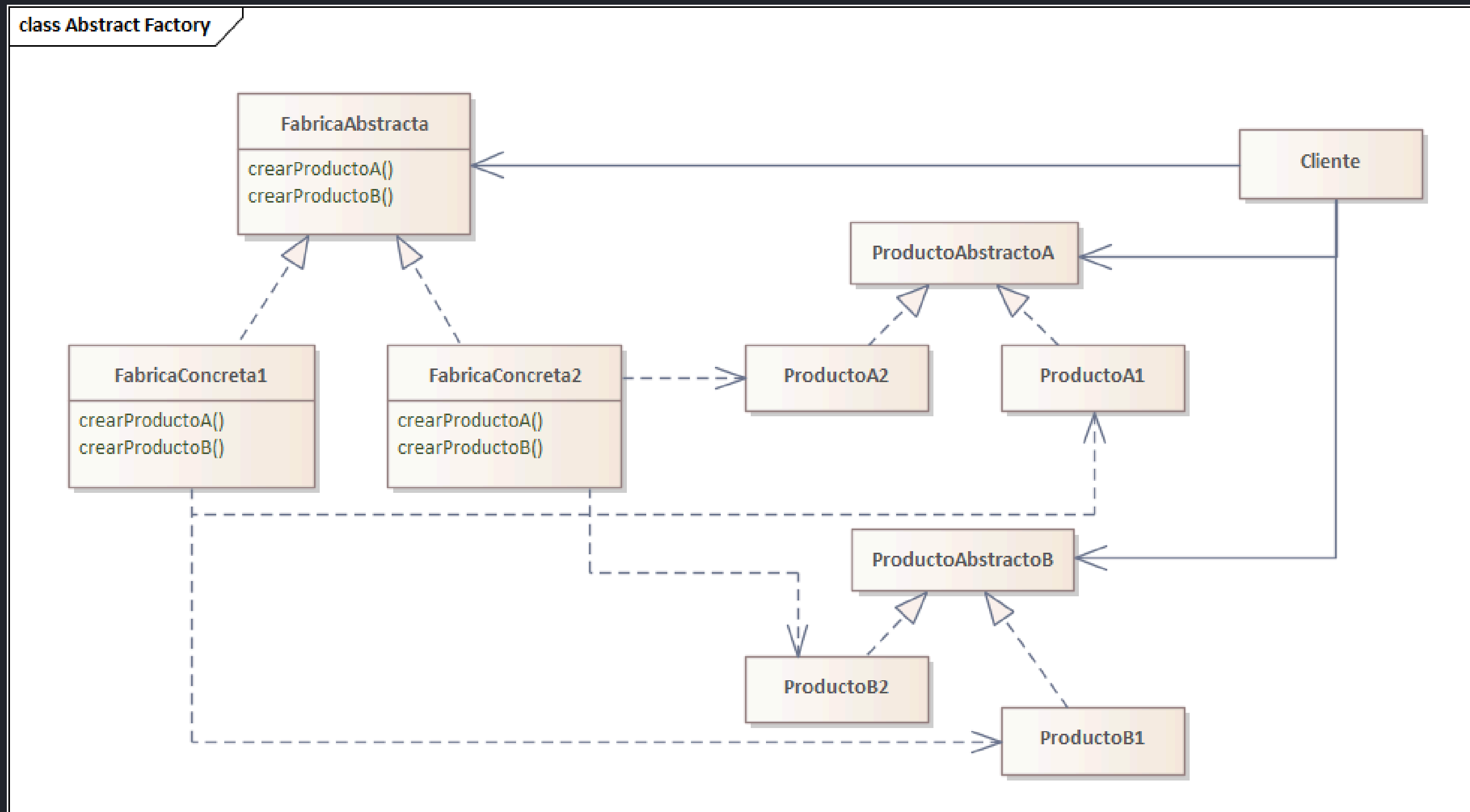


Diagrama basado en la estructura UML propuesta por GoF para el patrón Abstract Factory.

}

Colaboración entre participantes {

01

Se crea una Fabrica
concreta

02

La Fábrica concreta crea
Productos con una
implementacion determinada

03

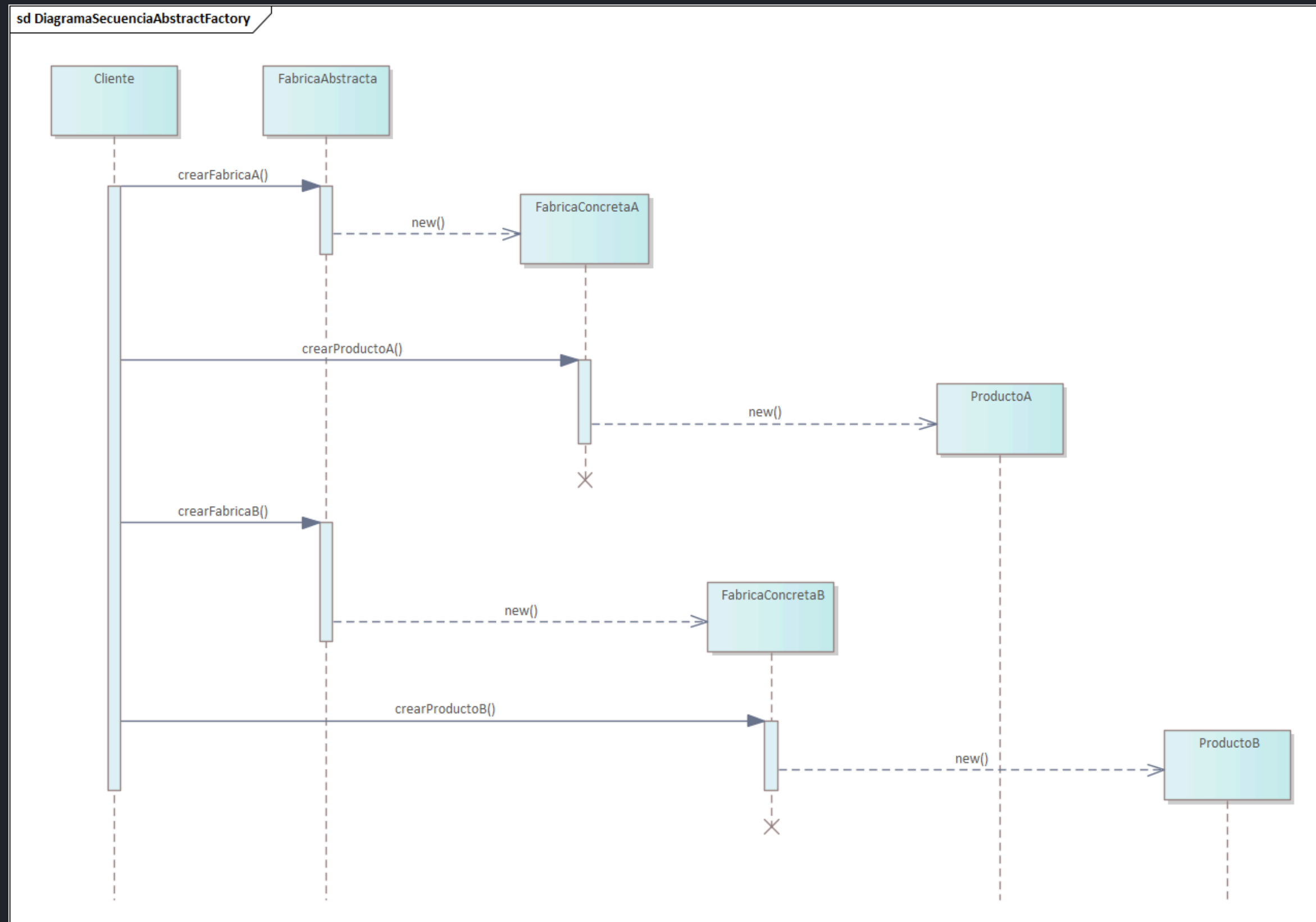
FabricaConcreta para
tipo de Productos

04

La Fábrica abstracta encarga la
creación de los productos a una
Fábrica concreta

}

Diagrama de Secuencia {



}

Consecuencias {

Ventajas {

01

Aislamiento de la implementación. El cliente no conoce el proceso de creación de los productos.

02

Facilita el cambio de familia de producto, sin afectar la implementación.

03

Existe consistencia entre los productos. Todos los objetos creados pertenecen a una misma familia.

}

}

Consecuencias {

Desventajas {

01

Dificulta la adición de nuevo tipo de productos.

02

Existe la posibilidad de que tenga una estructura grande de clases lo que dificultaría su mantenibilidad

03

Esta fuertemente orientado a familias de objetos. Introduce rigidez.

}

}

Cuándo conviene usarlo {

01

Cuando es fácil identificar familias de objetos.

02

Cuando se requiere crear objetos compatibles entre sí.

03

Se necesita aislar al cliente de las clases concretas.

04

Cuando debes cambiar fácilmente las familias de los productos.

}

Cuándo *no* conviene usarlo {

01

Cuando no se pueden formar familias.

02

Cuando se debe agregar muchos tipos de productos

03

Cuando se necesita crear objetos individuales simples.

04

Cuando se necesita un diseño simple y ligero

}

Patrones relacionados{

01 Factory Method

Abstract Factory puede implementarse con métodos de fabricación.

02 Prototype

Se apoya en este patrón al clonar objetos en vez de instanciar directamente.

03 Singleton

Se implementa como Singleton para que solo exista una fábrica concreta.

}

Relación con los Principios de diseño
{

01 Encapsular lo que varía

Encapsula la creación de diferentes familias de productos dentro de fábricas concretas.

02 Favorecer composición sobre herencia

Trabaja con objetos creados por fábricas, en lugar de depender de una jerarquía de herencia.

03 Programar para interfaces

El cliente trabaja con interfaces abstractas tanto para las fábricas como para los productos.

}

Implementación {

Planteamiento del problema {

Una empresa dedicada a la fabricación de vehículos desarrolla un sistema para crear automóviles y motocicletas.

Al inicio, el sistema solamente trabajaba con vehículos eléctricos, por lo que los objetos podían crearse directamente desde clases concretas.

Sin embargo, con el crecimiento del sistema surgió la necesidad de incorporar nuevas líneas de vehículos

}

}

Implementación {

Planteamiento del problema {

El problema aparece porque la lógica depende directamente de clases específicas.

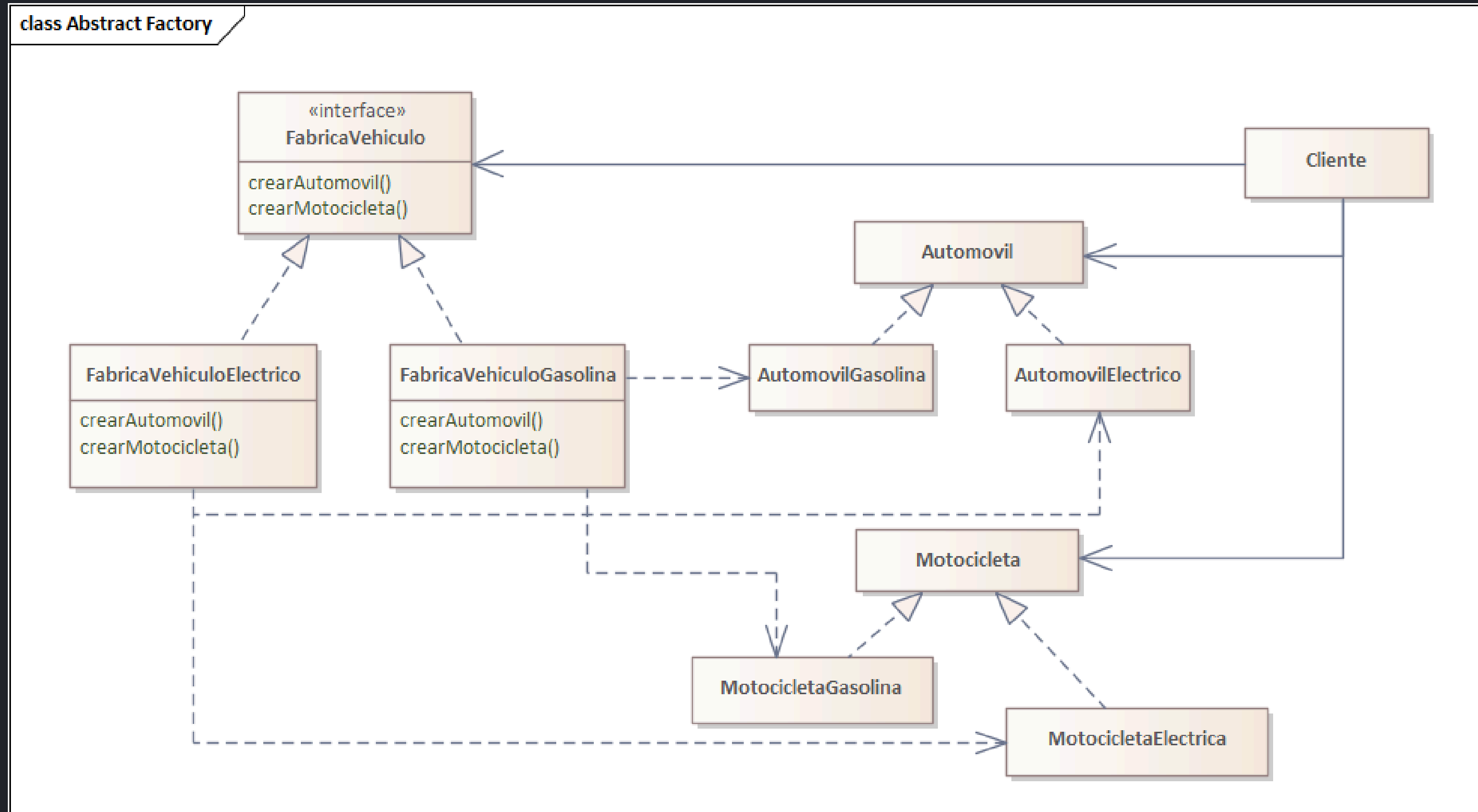
Esto provoca:

- alto acoplamiento entre clases
- baja mantenibilidad
- poca escalabilidad
- dificultad para extender el sistema
- violación del principio abierto/cerrado

}

}

Diagrama de Clases para la solución al problema {



}

Puntos esenciales de PHP {

Lenguaje de programación del lado del servidor.

PHP suele considerarse un lenguaje de baja a media complejidad.

Es fácil de aprender gracias a sintaxis accesible y a la gran cantidad de recursos disponibles.



}

Sintáxis de PHP {

PHP se puede usar como un lenguaje procedural (como C++) pero también funciona con un enfoque orientado a objetos (como Java).

```
class Persona {  
    public $nombre;  
    public $edad;  
  
    public function saludar() {  
        echo "Hola, soy " . $this->nombre;  
    }  
}
```

```
$persona = new Persona();  
$persona->nombre = "Vic";  
$persona->saludar();
```

}

Demostración {

```
Selecione una fabrica:  
1. Vehiculos electricos  
2. Vehiculos de gasolina  
1
```

```
Automovil:  
Automovil electrico  
Modelo: Tesla Model X  
Color: Rojo  
Potencia: 500  
Espacio: 5.5
```

```
Motocicleta:  
Motocicleta electrica  
Modelo: Yamaha MT-07  
Color: Negro  
Potencia: 300
```

```
Selecione una fabrica:  
1. Vehiculos electricos  
2. Vehiculos de gasolina  
2
```

```
Automovil:  
Automovil de gasolina  
Modelo: Tesla Model X  
Color: Rojo  
Potencia: 500  
Espacio: 5.5
```

```
Motocicleta:  
Motocicleta de gasolina  
Modelo: Yamaha MT-07  
Color: Negro  
Potencia: 300
```

}

Segundo principio {

Builder

}

Problema

El problema que resuelve el patrón Builder Pattern radica en la construcción de objetos complejos que requieren múltiples pasos o configuraciones específicas.

Cuando surge la necesidad de crear diferentes representaciones del mismo objeto, la solución inicial comienza a volverse difícil de mantener debido a la concentración de toda la lógica de construcción en una sola clase.



Recurso gráfico utilizado como apoyo visual.

Solución {

Para resolver esto, el patrón Builder separa el proceso de construcción del objeto final, permitiendo construir distintas variantes paso a paso sin modificar la lógica principal del sistema.

}

Patrón {

Nombre Oficial
Builder o Constructor

Clasificación
Creacional

Alcance
Objeto

Intención del patrón

“Separar la construcción de un objeto complejo de su representación, de forma que un mismo proceso de construcción pueda crear diferentes representaciones.”
Gamma et al., 2003, p. 89.

}

Estructura {

Responsabilidades de cada parte {

- **Constructor** : Define una interfaz abstracta para la creación de las partes de un Objeto.
- **Constructor concreto** : Implementa la interfaz abstracta para construir y ensamblar las partes del Producto
- **Director** : Es una clase que controla el proceso de construcción siguiendo una secuencia u orden específico
- **Producto** : Es el objeto construido resultante.
- **Cliente** : Utiliza el Director o directamente el Constructor concreto para construir el objeto

}

}

Estructura {

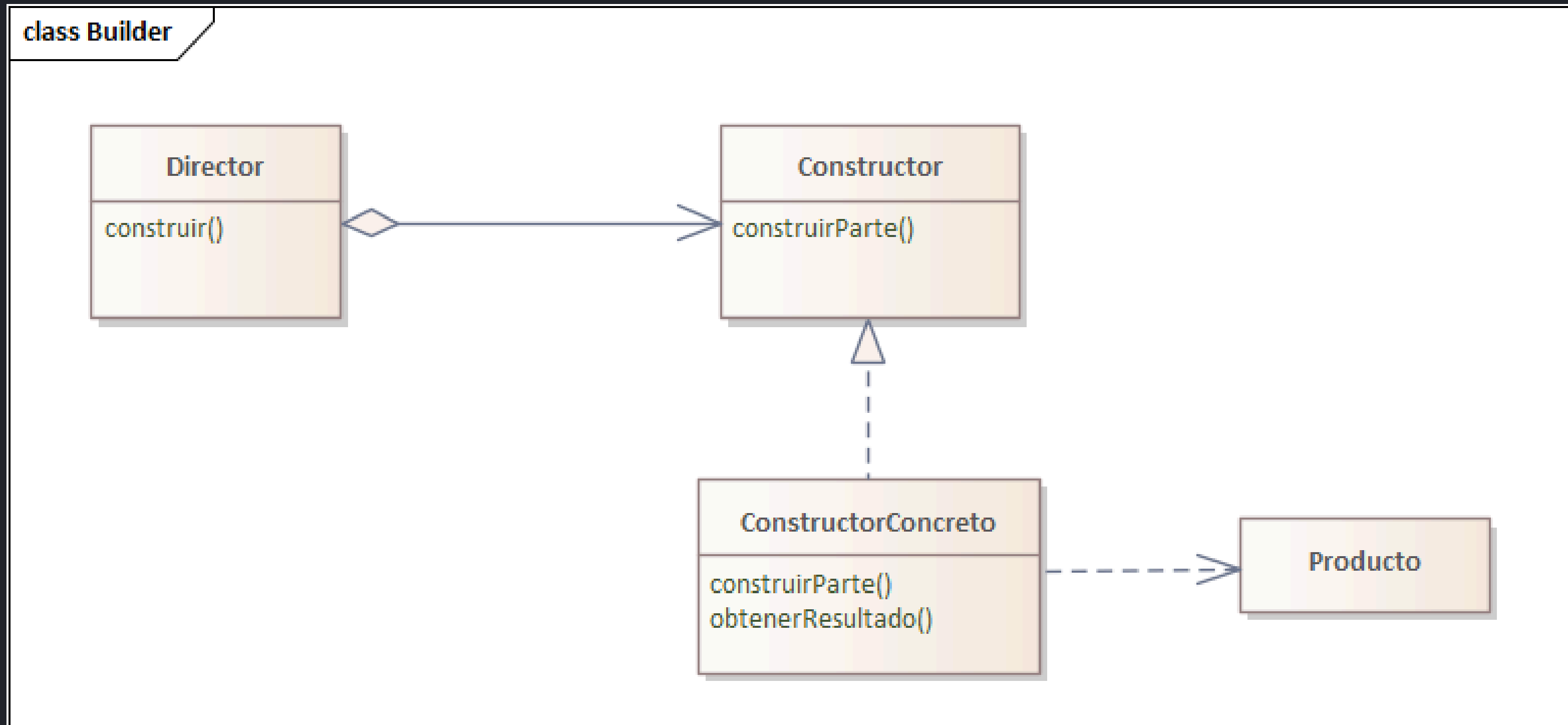


Diagrama basado en la estructura UML propuesta por GoF para el patrón Builder.

}

Colaboración entre participantes {

01

El Cliente crea un
Director con el
Constructor deseado

02

El Director notifica al
Constructor cada vez que se
construye una parte.

03

El Constructor atiende
las peticiones del
Director y las añade.

04

El Cliente obtiene el Producto
del Constructor

}

Diagrama de Secuencia {

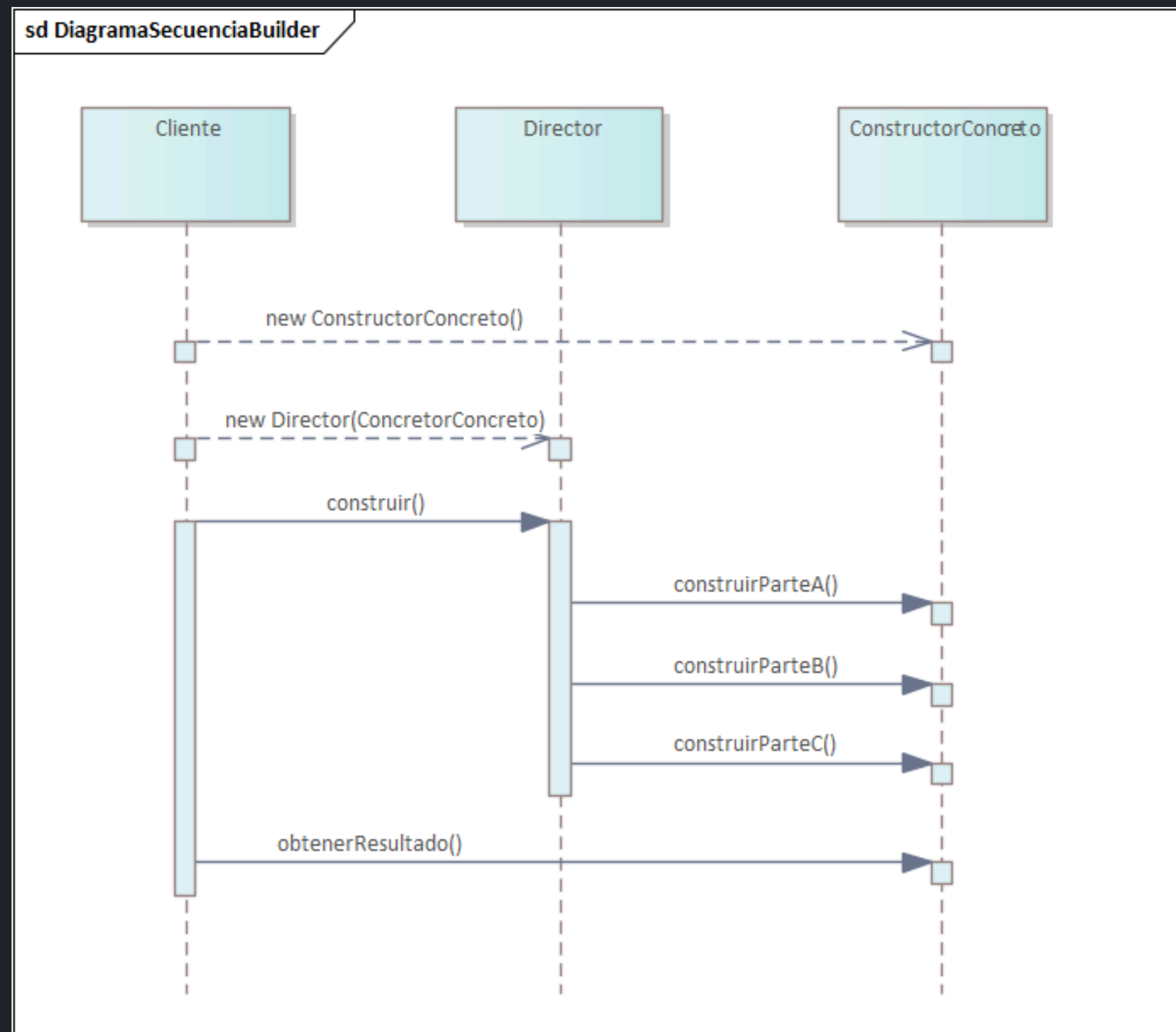


Diagrama basado en la estructura UML propuesta por GoF para el patrón Builder.

}

Consecuencias {

Ventajas {

01

Permite reutilizar el mismo proceso de construcción para generar muchas variantes de un producto.

02

Crea objetos complejos paso a paso, mejorando el control sobre la creación de objetos.

03

Mejora la legibilidad al evitar los constructores extensos.

}

}

Consecuencias {

Desventajas {

01

Incrementa la complejidad del diseño del sistema.

02

Requiere mayor tiempo de implementación en comparación con un constructor normal.

03

Si el proceso de construcción cambia constantemente, puede ser necesario modificar muchos constructores.

}

}

Cuándo conviene usarlo {

01

Cuando se necesitan construir objetos complejos paso a paso.

02

Cuando un objeto tiene muchos parámetros opcionales o configurables.

03

Cuando se debe producir diferentes versiones de un objeto

04

Cuando se quiere evitar constructores largos o confusos.

}

Cuándo *no* conviene usarlo {

01

Cuando el objeto es simple y se puede construir con un constructor normal.

02

Cuando no hay un proceso de construcción complejo o por pasos.

03

Cuando no hay variaciones importantes en la forma de construir el objeto.

04

Cuando la simplicidad del sistema es más importante que la flexibilidad.

}

Patrones relacionados{

01 Abstract Factory

Ambos pueden construir objetos complejos.

Mientras Builder construye el objeto paso a paso, Abstract Factory se enfoca en familias de objetos producto.

Builder devuelve el producto como paso final, mientras que Abstract Factory lo devuelve inmediatamente.

}

Relación con los Principios de diseño
{

01 Encapsular lo que varía

El proceso de construcción puede variar sin afectar al cliente.

02 Favorecer composición sobre herencia

Permite construir variantes usando constructores sin necesidad de crear jerarquías de herencia.

03 Programar para interfaces

El cliente trabaja con la interfaz Builder en lugar de depender de clases concretas.

}

Implementación {

Planteamiento del problema {

Una constructora desarrolla un sistema para generar distintos tipos de casas. Cada casa requiere varias partes durante su construcción, como paredes, techo, puertas...

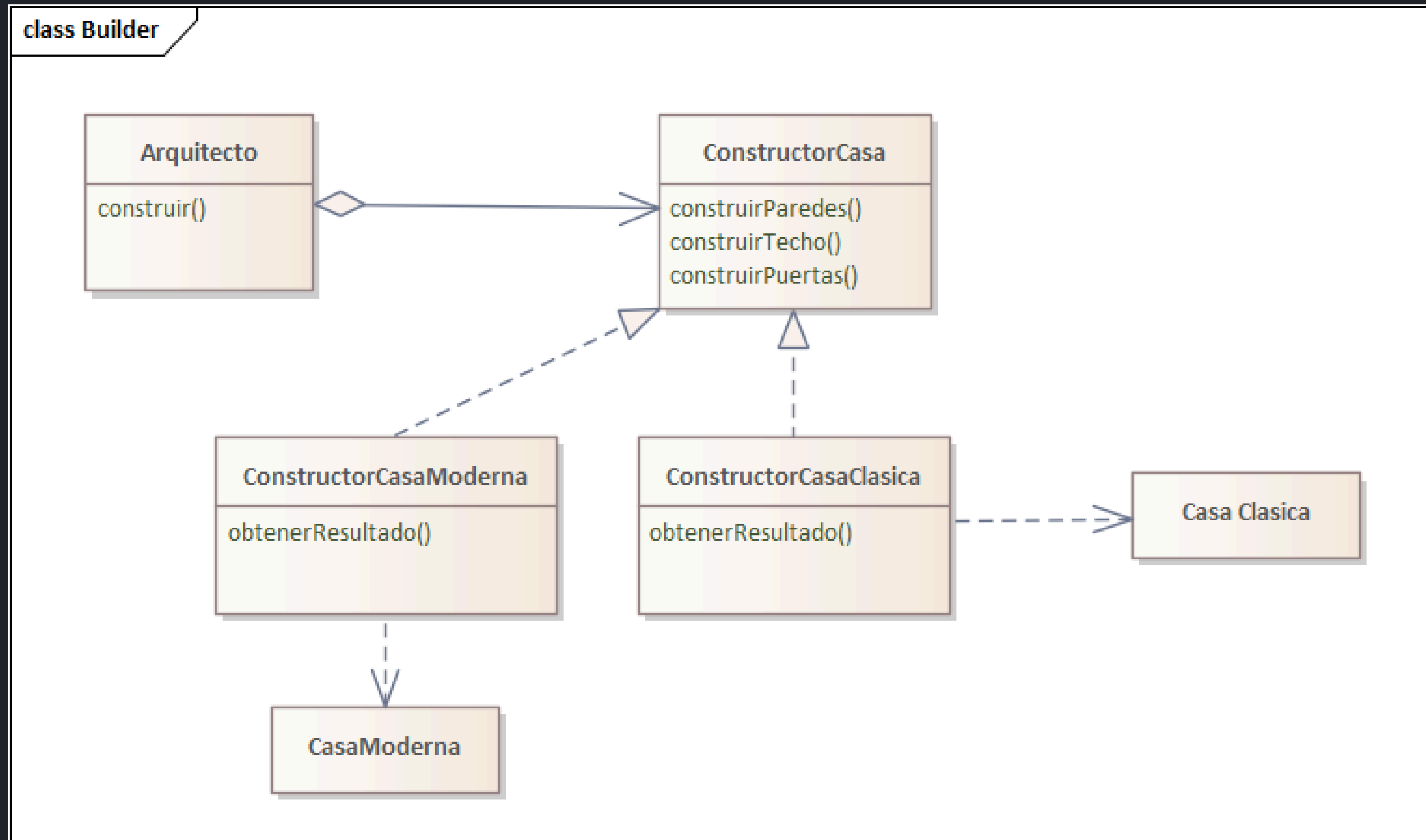
El problema surge porque cada tipo de casa requiere diferentes formas de construcción, provocando:

- duplicación de código
- baja reutilización
- poca flexibilidad
- dificultad para agregar nuevas variantes
- alta complejidad en la construcción de objetos

}

}

Diagrama de Clases {



}

Demostración {

Desea construir una Casa Moderna (1) o Clasica (2)?: 1

Casa moderna

Paredes modernas de Concreto

Techo moderno de Concreto

Puertas minimalistas de Concreto

Desea construir una Casa Moderna (1) o Clasica (2)?: 2

Casa clásica

Paredes clasicas de Concreto

Techo clasico de Concreto

Puertas de madera de Concreto

}

Referencias Bibliograficas {

Debrauwer, L. (2023). Patrones de diseño en Java: Los 23 modelos de diseño. Ediciones ENI.
https://www.gob.mx/cms/uploads/attachment/file/1001640/Patrones_de_diseno_java_modelos.pdf

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (2003). Patrones de diseño: Elementos de software orientado a objetos reutilizable. Pearson Educación.

GeeksforGeeks. (2026, 21 enero). Builder design pattern. GeeksforGeeks.
<https://www.geeksforgeeks.org/system-design/builder-design-pattern/>

}


```
<!--Universidad Veracruzana-->
```

Gracias {

```
<Por="Equipo 8"/>
```

}